

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

Кафедра «Вычислительные системы и технологии»

ОТЧЕТ

по лабораторной работе №1

по дисциплине

Шаблоны проектирования программного обеспечения

ЛР1 Реализация паттернов проектирования программного обеспечения
средствами языка Java

РУКОВОДИТЕЛЬ:

_____ Сапожников В.О.

СТУДЕНТЫ:

_____ Сигаев В. Ю.

_____ 24-ИВТ-3

Работа защищена «__» _____

С оценкой _____

Нижний Новгород 2026

Задание:

Разработайте объектно-ориентированную модель музыкального плеера. Плеер управляет коллекцией треков, каждый из которых характеризуется названием, исполнителем, жанром, длительностью и битрейтом. Система должна поддерживать:

1. Создание и редактирование плейлистов, включая вложенные плейлисты (плейлист может содержать другие плейлисты).
2. Различные режимы воспроизведения (последовательный, случайный, повтор одного трека, повтор плейлиста).
3. Управление воспроизведением (play, pause, stop, next, previous) с отображением текущего состояния.
4. Фильтрацию и сортировку треков по различным критериям (жанр, исполнитель, длительность).

Система должна быть расширяема по режимам воспроизведения, форматам треков и источникам музыки.

Паттерны:

1. Компоновщик (Composite)

Классы: MusicComponent(интерфейс), MusicContainer(интерфейс), Playlist, TrackLeaf

Почему использован: Музыкальная библиотека имеет иерархическую структуру. Плейлист может содержать как отдельные треки, так и другие плейлисты (подборки).

Обоснование: Паттерн позволяет обращаться к единичному треку (TrackLeaf) и к целому плейлисту (Playlist) одинаково через общий интерфейс MusicComponent. Это упрощает код: плееру не нужно знать, проигрывает он сейчас один файл или вложенную папку с музыкой.

2. Команда (Command)

Классы: Command(интерфейс), NextCommand, PauseCommand, PlayCommand, PreviousCommand, StopCommand

Почему использован: Для реализации кнопок управления плеером (Play, Stop, Next).

Обоснование: Вместо того чтобы прописывать всю логику внутри класса Main или напрямую в кнопках, каждая операция вынесена в отдельный класс-команду.

- **Гибкость:** Мы можем легко добавить новые команды (например, «Запись» или «Таймер сна»), не меняя код самого плеера.
- **Отложенный запуск:** Команды можно складывать в очередь или сохранять историю для отмены действий.

3. Стратегия (Strategy)

Классы: PlayStrategy(интерфейс), RandomStrategy, RepeatAllStrategy, RepeatOneStrategy, SequentialStrategy, TrackSortStrategy(интерфейс), SortByDuration

Почему использован: Для управления режимами воспроизведения (последовательно, случайно, повтор).

Обоснование: Алгоритм выбора «следующего трека» может меняться прямо во время работы программы. Паттерн позволяет вынести эти алгоритмы в отдельные классы. Плеер просто вызывает метод «дай следующий трек», а какая именно стратегия сейчас активна (RandomStrategy или SequentialStrategy), решает пользователь.

4. Состояние (State)

Классы: PlayerState(интерфейс), PausedState, StoppedState, PlayingState

Почему использован: Поведение плеера кардинально меняется в зависимости от его текущего статуса (Играет, На паузе, Остановлен).

Обоснование: Если использовать обычные if-else, код превратится в «спагетти» (например, если нажата кнопка Play, а статус «Пауза» — делаем одно, если статус «Стоп» — другое). Паттерн Состояние делегирует это поведение объектам состояний. Это делает логику переходов между состояниями чистой и предсказуемой.

5. Строитель (Builder)

Класс: Track

Почему использован: Класс Track имеет много полей (название, артист, жанр, длительность, битрейт).

Обоснование: Использование конструктора с 5-7 параметрами неудобно и приводит к ошибкам (легко перепутать две идущие подряд цифры или строки). Билдер позволяет создавать объект пошагово, делая код читаемым (.setTitle("...").setDuration(...)) и защищая объект от создания в неполном или некорректном виде.

Пример работы:

```
Музыкальный плеер
1. Создать плейлист (в корень)
2. Создать вложенный плейлист
3. Добавить трек в плейлист
4. Показать структуру плейлистов
5. Выбрать плейлист
6. Воспроизвести
7. Пауза
8. Стоп
9. Следующий
10. Предыдущий
11. Последовательный режим
12. Случайный режим
13. Повторять один трек
14. Повторять весь плейлист
15. Сортировка по длительности
0. Выход
4
```

```
Структура библиотеки:
```

```
Плейлист: Main
```

```
    Плейлист: рок
```

```
        Трек: рок 1
```

```
        Трек: рок 2
```

Листинг:

Command:

```
package command;
```

```
public interface Command {
    void execute();
}
```

NextCommand

```
package command;
```

```
import player.Player;
```

```
public class NextCommand implements Command {
```

```

private Player player;

public NextCommand(Player player) {
    this.player = player;
}

@Override
public void execute() {
    player.next();
}
}

```

PauseCommand

```

package command;

import player.Player;

public class PauseCommand implements Command {

    private Player player;

    public PauseCommand(Player player) {
        this.player = player;
    }

    @Override
    public void execute() {
        player.pause();
    }
}

```

PlayCommand

```

package command;

import player.Player;

public class PlayCommand implements Command {

    private Player player;

    public PlayCommand(Player player) {
        this.player = player;
    }
}

```

```

    }

    @Override
    public void execute() {
        player.play();
    }
}

```

PreviousCommand

```

package command;

import player.Player;

public class PreviousCommand implements Command {

    private Player player;

    public PreviousCommand(Player player) {
        this.player = player;
    }

    @Override
    public void execute() {
        player.previous();
    }
}

```

StopCommand

```

package command;

import player.Player;

public class StopCommand implements Command {

    private Player player;

    public StopCommand(Player player) {
        this.player = player;
    }

    @Override
    public void execute() {

```

```
        player.stop();
    }
}
```

MusicComponent

```
package composite;
```

```
import model.Track;
import java.util.List;
```

```
public interface MusicComponent {
    void play();
    void showStructure(String indent);

    default void collectTracks(List<Track> trackList) {}

    default Playlist findPlaylist(String name) {
        return null;
    }
}
```

MusicContainer

```
package composite;
```

```
public interface MusicContainer extends MusicComponent {
    Playlist findPlaylist(String name);
}
```

```
package composite;
```

```
import model.Track;
```

```
import java.util.ArrayList;
import java.util.List;
```

```
public class Playlist implements MusicComponent {
    private String name;
    private List<MusicComponent> components = new ArrayList<>();
}
```

```

public Playlist(String name) {
    this.name = name;
}

public void add(MusicComponent component) {
    components.add(component);
}

@Override
public void play() {
    System.out.println("Плейлист: " + name + " ");
    for (MusicComponent c : components) {
        c.play();
    }
}

public List<Track> getAllTracks() {
    List<Track> allTracks = new ArrayList<>();
    collectTracks(allTracks);
    return allTracks;
}

@Override
public void collectTracks(List<Track> trackList) {
    for (MusicComponent c : components) {
        c.collectTracks(trackList);
    }
}

@Override
public void showStructure(String indent) {
    System.out.println(indent + "Плейлист: " + name);
    for (MusicComponent c : components) {
        c.showStructure(indent + " ");
    }
}

@Override
public Playlist findPlaylist(String name) {
    if (this.name.equalsIgnoreCase(name)) {
        return this;
    }
}

```



```

        for (MusicComponent c : components) {
            Playlist found = c.findPlaylist(name);
            if (found != null) {
                return found;
            }
        }
        return null;
    }
}

```

TrackLeaf

```

package composite;

import model.Track;

import java.util.List;

public class TrackLeaf implements MusicComponent {
    private final Track track;

    public TrackLeaf(Track track) {
        this.track = track;
    }

    @Override
    public void play() {
        System.out.println("Играет: " + track.getTitle());
    }

    @Override
    public void showStructure(String indent) {
        System.out.println(indent + "Трек: " + track.getTitle());
    }

    @Override
    public void collectTracks(List<Track> trackList) {
        trackList.add(this.track);
    }
}

```

Track

package model;

```
public class Track {
    private String title;
    private String artist;
    private String genre;
    private int duration;
    private int bitrate;

    private Track(Builder builder) {
        this.title = builder.title;
        this.artist = builder.artist;
        this.genre = builder.genre;
        this.duration = builder.duration;
        this.bitrate = builder.bitrate;
    }

    public String getTitle() {
        return title;
    }

    public String getArtist() {
        return artist;
    }

    public String getGenre() {
        return genre;
    }

    public int getDuration() {
        return duration;
    }

    public int getBitrate() {
        return bitrate;
    }

    public static class Builder {
        private String title;
        private String artist;
```

```

private String genre;
private int duration;
private int bitrate;

public Builder setTitle(String title) {
    this.title = title;
    return this;
}

public Builder setArtist(String artist) {
    this.artist = artist;
    return this;
}

public Builder setGenre(String genre) {
    this.genre = genre;
    return this;
}

public Builder setDuration(int duration) {
    this.duration = duration;
    return this;
}

public Builder setBitrate(int bitrate) {
    this.bitrate = bitrate;
    return this;
}

public Track build() {
    return new Track(this);
}
}

```

Player

```
package player;
```

```
import java.util.List;
import model.Track;
```

```
import strategy.PlayStrategy;
import state.PlayerState;
import sort.TrackSortStrategy;

public class Player {
    private PlayerState state;
    private PlayStrategy strategy;
    private List<Track> tracks;
    private int currentIndex;
    private TrackSortStrategy sortStrategy;

    public void play() {
        state.play(this);
    }

    public void next() {
        Track nextTrack = strategy.next(tracks, currentIndex);
        currentIndex = tracks.indexOf(nextTrack);
        System.out.println("Сейчас играет: " + nextTrack.getTitle());
    }

    public void setTracks(List<Track> tracks) {
        this.tracks = tracks;
        this.currentIndex = 0;
    }

    public void setState(PlayerState state) {
        this.state = state;
    }

    public void setStrategy(PlayStrategy strategy) {
        this.strategy = strategy;
    }

    public void setSortStrategy(TrackSortStrategy sortStrategy) {
        this.sortStrategy = sortStrategy;
    }

    public void pause() {
        state.pause(this);
    }
}
```

```

public void stop() {
    state.stop(this);
}

public void previous() {

    if (tracks.isEmpty()) {
        return;
    }

    currentIndex--;

    if (currentIndex < 0) {
        currentIndex = tracks.size() - 1;
    }

    Track track = tracks.get(currentIndex);

    System.out.println("Сейчас играет: " + track.getTitle());
}

public void sortCurrentPlaylist() {
    if (sortStrategy == null) return;

    sortStrategy.sort(tracks);

    System.out.println("Треки отсортированы");
}
}

```

SortByDuration

```

package sort;

import model.Track;

import java.util.List;
import java.util.Comparator;

public class SortByDuration implements TrackSortStrategy {
    public void sort(List<Track> tracks) {
        tracks.sort(Comparator.comparingInt(Track::getDuration));
    }
}

```

```
    }  
}
```

TrackSortStrategy

```
package sort;
```

```
import model.Track;
```

```
import java.util.List;
```

```
public interface TrackSortStrategy {  
    void sort(List<Track> tracks);  
}
```

PausedState

```
package state;
```

```
import player.Player;
```

```
class PausedState implements PlayerState {
```

```
    public void play(Player player) {  
        System.out.println("Возобновление...");  
        player.setState(new PlayingState());  
    }
```

```
    public void pause(Player player) {  
        System.out.println("Трек уже на паузе");  
    }
```

```
    public void stop(Player player) {  
        System.out.println("Останавливаю с паузы...");  
        player.setState(new StoppedState());  
    }
```

```
}
```

PlayerState

```
package state;
```

```
import player.Player;
```

```
public interface PlayerState {
```

```
void play(Player player);  
void pause(Player player);  
void stop(Player player);  
}
```

PlayingState

```
package state;  
  
import player.Player;  
  
public class PlayingState implements PlayerState {  
  
    public void play(Player player) {  
        System.out.println("Уже играет");  
    }  
  
    public void pause(Player player) {  
        System.out.println("Пауза...");  
        player.setState(new PausedState());  
    }  
  
    public void stop(Player player) {  
        System.out.println("Остановка...");  
        player.setState(new StoppedState());  
    }  
}
```

StoppedState

```
package state;  
  
import player.Player;  
  
public class StoppedState implements PlayerState {  
  
    public void play(Player player) {  
        System.out.println("Начинаю играть...");  
        player.setState(new PlayingState());  
    }  
  
    public void pause(Player player) {  
        System.out.println("Нельзя поставить на паузу, трек остановлен");  
    }  
}
```

```
    public void stop(Player player) {  
        System.out.println("Трек уже остановлен");  
    }  
}
```

PlayStrategy

```
package strategy;  
  
import model.Track;  
  
import java.util.List;  
  
public interface PlayStrategy {  
    Track next(List<Track> tracks, int currentIndex);  
}
```

RandomStrategy

```
package strategy;  
  
import model.Track;  
  
import java.util.List;  
import java.util.Random;  
  
public class RandomStrategy implements PlayStrategy {  
    private Random random = new Random();  
  
    public Track next(List<Track> tracks, int currentIndex) {  
        return tracks.get(random.nextInt(tracks.size()));  
    }  
}
```

RepeatAllStrategy

```
package strategy;  
  
import model.Track;  
  
import java.util.List;  
  
public class RepeatAllStrategy implements PlayStrategy {  
    public Track next(List<Track> tracks, int currentIndex) {
```



```

        return tracks.get((currentIndex + 1) % tracks.size());
    }
}

```

RepeatOneStrategy

```

package strategy;

import model.Track;

import java.util.List;

public class RepeatOneStrategy implements PlayStrategy {
    public Track next(List<Track> tracks, int currentIndex) {
        return tracks.get(currentIndex);
    }
}

```

SequentialStrategy

```

package strategy;

import model.Track;

import java.util.List;

public class SequentialStrategy implements PlayStrategy {
    public Track next(List<Track> tracks, int currentIndex) {
        return tracks.get((currentIndex + 1) % tracks.size());
    }
}

```

Main

```

import command.*;
import composite.*;
import model.Track;
import player.Player;
import state.StoppedState;
import strategy.*;
import sort.SortByDuration;

import java.util.*;

public class Main {

```

```
public static void main(String[] args) {

    Scanner scanner = new Scanner(System.in);

    Playlist rootPlaylist = new Playlist("Main");
    Playlist currentPlaylist = rootPlaylist;

    Player player = new Player();
    player.setState(new StoppedState());
    player.setStrategy(new SequentialStrategy());

    player.setTracks(currentPlaylist.getAllTracks());

    boolean running = true;

    while (running) {
        System.out.println("\nМузыкальный плеер");
        System.out.println("1. Создать плейлист (в корень)");
        System.out.println("2. Создать вложенный плейлист");
        System.out.println("3. Добавить трек в плейлист");
        System.out.println("4. Показать структуру плейлистов");
        System.out.println("5. Выбрать плейлист");
        System.out.println("6. Воспроизвести");
        System.out.println("7. Пауза");
        System.out.println("8. Стоп");
        System.out.println("9. Следующий");
        System.out.println("10. Предыдущий");
        System.out.println("11. Последовательный режим");
        System.out.println("12. Случайный режим");
        System.out.println("13. Повторять один трек");
        System.out.println("14. Повторять весь плейлист");
        System.out.println("15. Сортировка по длительности");
        System.out.println("0. Выход");

        int choice = scanner.nextInt();
        scanner.nextLine();

        switch (choice) {
            case 1:
                System.out.print("Имя плейлиста: ");
                String playlistName = scanner.nextLine();
```

```
Playlist newPlaylist = new Playlist(playlistName);
rootPlaylist.add(newPlaylist);
System.out.println("Плейлист создан в корне");
break;
```

case 2:

```
System.out.print("Имя родительского плейлиста: ");
String parentName = scanner.nextLine();
Playlist parent = rootPlaylist.findPlaylist(parentName);
```

```
if (parent == null) {
    System.out.println("Плейлист не найден");
    break;
}
```

```
System.out.print("Имя нового вложенного плейлиста: ");
String nestedName = scanner.nextLine();
parent.add(new Playlist(nestedName));
System.out.println("Вложенный плейлист добавлен");
break;
```

case 3:

```
System.out.print("В какой плейлист добавить трек?: ");
String targetName = scanner.nextLine();
Playlist target = rootPlaylist.findPlaylist(targetName);
```

```
if (target == null) {
    System.out.println("Плейлист не найден");
    break;
}
```

```
System.out.print("Название: "); String title = scanner.nextLine();
System.out.print("Исполнитель: "); String artist = scanner.nextLine();
System.out.print("Жанр: "); String genre = scanner.nextLine();
System.out.print("Продолжительность (сек): "); int duration =
scanner.nextInt();
System.out.print("Битрейт: "); int bitrate = scanner.nextInt();
scanner.nextLine();
```

```
Track track = new Track.Builder()
    .setTitle(title)
    .setArtist(artist)
```

```
.setGenre(genre)
.setDuration(duration)
.setBitrate(bitrate)
.build();
```

```
target.add(new TrackLeaf(track));
```

```
player.setTracks(currentPlaylist.getAllTracks());
System.out.println("Трек успешно добавлен");
break;
```

case 4:

```
System.out.println("\nСтруктура библиотеки.");
rootPlaylist.showStructure("");
break;
```

case 5:

```
System.out.print("Введите имя плейлиста для переключения: ");
String selectedName = scanner.nextLine();
Playlist selected = rootPlaylist.findPlaylist(selectedName);
```

```
if (selected == null) {
    System.out.println("Плейлист не найден");
    break;
}
```

```
currentPlaylist = selected;
player.setTracks(currentPlaylist.getAllTracks());
System.out.println("Переключено на плейлист: " + selectedName);
break;
```

```
case 6: new PlayCommand(player).execute(); break;
case 7: new PauseCommand(player).execute(); break;
case 8: new StopCommand(player).execute(); break;
case 9: new NextCommand(player).execute(); break;
case 10: new PreviousCommand(player).execute(); break;
```

case 11:

```
player.setStrategy(new SequentialStrategy());
System.out.println("Режим: Последовательно");
break;
```

```

case 12:
    player.setStrategy(new RandomStrategy());
    System.out.println("Режим: Случайно");
    break;

case 13:
    player.setStrategy(new RepeatOneStrategy());
    System.out.println("Режим: Повтор одного трека");
    break;

case 14:
    player.setStrategy(new RepeatAllStrategy());
    System.out.println("Режим: Повтор плейлиста");
    break;

case 15:
    player.setSortStrategy(new SortByDuration());
    player.sortCurrentPlaylist();
    System.out.println("\nОтсортированные треки в текущем
плейлисте:");
    for (Track t : currentPlaylist.getAllTracks()) {
        System.out.println(t.getTitle() + " [" + t.getDuration() + " сек]");
    }
    break;

case 0:
    running = false;
    break;

default:
    System.out.println("Неверный ввод");
}
}
}
}
}

```